

Azure SOC Lab Guide

AUTOMATED LINUX BRUTE-FORCE DETECTION USING MICROSOFT SENTINEL & DEFENDER FOR CLOUD

This deployment framework outlines the end-to-end practical steps to ingest, parse, aggregate, and alert on malicious Linux authentication failures inside Azure. By shifting from legacy architectures to the unified **Microsoft Defender / Sentinel Platform**, this guide leverages the modern **Azure Monitor Agent (AMA)** and **Data Collection Rules (DCR)** to achieve near real-time security monitoring.

Phase 1: Environment Provisioning & Base Infrastructure

Before monitoring ingestion workflows, ensure your base lab assets are correctly initialized in a synchronized Azure region.

1. Virtual Machine Deployment

- Provision an **Ubuntu Server VM** (e.g., Ubuntu 20.04 or 22.04 LTS).
- Ensure it is placed in a designated Resource Group (e.g., **LAB**).
- Maintain an active Public IP configuration initially to simulate inbound login traffic from external endpoints.

2. Provision Log Analytics Workspace (LAW)

The workspace acts as the central ingestion telemetry vault where raw security logs are indexed and queried.

1. Navigate to the **Azure Portal** → Search for **Log Analytics Workspaces** → Click **Create**.
2. Configure the initialization parameters with the exact settings below:
 - **Name:** **LAW-SOC-LAB**
 - **Region:** **Central India**
 - **Resource Group:** Select or match your active lab group (e.g., **LAB**).
3. Click **Review + Create**, then confirm with **Create**.

ARCHITECTURAL INSIGHT:

Always align your Log Analytics Workspace and the target Virtual Machines in the same geographical Azure region (e.g., Central India) whenever possible. This minimizes cross-region data egress costs and eliminates processing latency for high-throughput security event routing.

Phase 2: Azure Monitor Agent (AMA) Management via Azure CLI

The legacy Log Analytics agent (OMS) is deprecated. We utilize the newer **Azure Monitor Agent (AMA)**, which offers localized filtering capabilities using Data Collection Rules.

1. Target Environment Discovery

Open your Azure CLI environment (Cloud Shell or local terminal) and execute the discovery layout to ensure target host mapping:

```
az vm list -o table
```

2. Extension Installation Execution

Inject the localized native `AzureMonitorLinuxAgent` extension directly into the running instance by executing the deployment configuration below:

```
az vm extension set --name AzureMonitorLinuxAgent --publisher  
Microsoft.Azure.Monitor --vm-name LinuxVM --resource-group LAB
```

3. Installation Verification

To audit deployment execution state and confirm the extension is provisioned successfully, query the extension runtime profile:

```
az vm extension list --vm-name LinuxVM --resource-group LAB --o table
```

4. In-Guest Operating System Validation

Establish a baseline terminal connection via SSH to your `LinuxVM` and confirm both the monitoring agent and internal platform agent daemons are in an active operational state:

```
sudo systemctl status azuremonitoragent  
sudo systemctl status wlinuxagent
```

DEPLOYMENT WARNING:

If `azuremonitoragent` shows a failed or inactive status, ensure your VM has outbound connectivity to Azure management endpoints. The AMA requires a direct path or via an enterprise proxy to pull configuration chunks successfully.

Phase 3: Telemetry Routing via Data Collection Rule (DCR)

Data Collection Rules (DCR) explicitly instruct the Azure Monitor Agent on which internal operating system facilities to parse and where to stream the resulting payloads.

1. Create the DCR Pipeline

1. Navigate to **Azure Portal** → Search for **Monitor** → Select **Data Collection Rules**.

2. Click **Create** and input the baseline identity rules:
 - **Rule Name:** DCR-LinuxVM-Syslog
 - **Region:** Central India
3. **Resources Tab:** Click + **Add resources**, discover and select your target LinuxVM .
4. **Collect Tab:** Click + **Add data source** and configure the syslog parameters:
 - **Data Source Type:** Linux Syslog
 - **LOG_AUTH Facility Minimum Level:** LOG_DEBUG
 - **LOG_AUTHPRIV Facility Minimum Level:** LOG_DEBUG
5. **Destination Tab:** Click + **Add destination**, pick **Azure Monitor Logs**, and select your provisioned workspace: LAW-SOC-LAB .
6. Click **Review + Create**, then commit the configuration by choosing **Save**.

2. Trigger Synthetic Log Injection & Local Audit

To bypass natural timing windows and force immediate replication stream health checks, execute a customized log entry generation command directly on the Ubuntu VM terminal:

```
logger "SENTINEL_TEST_123"
```

Verify that the configuration cache chunks have updated locally on the server:

```
sudo ls /etc/opt/microsoft/azuremonitoragent/config-cache/configchunks/
```

3. Validate Ingestion Status via KQL Queries

Return to the **Azure Portal**, select your **Log Analytics Workspace** (LAW-SOC-LAB), and open the **Logs** blade. Run the following test diagnostics to confirm pipeline data delivery:

Test 1: Search for Synthetic Canary Event

```
Syslog
| where SyslogMessage contains "SENTINEL_TEST_123"
```

Test 2: Check Machine Operational Heartbeat

```
Heartbeat
| where Computer == "LinuxVM"
| sort by TimeGenerated desc
```

Phase 4: SIEM Incident & Scheduled Alert Integration (Microsoft Sentinel)

Once logs flow predictably, we elevate raw data tracking into an intelligent security correlation policy within Microsoft Sentinel (now consolidated into Defender for Cloud Security Operations).

1. Analytics Rule Implementation

1. Open **Microsoft Sentinel** (or Unified Defender Portal) → Go to **Configuration** → **Analytics**.
2. Click **Create** → Select **Scheduled query rule**.
3. Provide the following parameter mapping details under the **General** tab:
 - **Name:** `Ubuntu Linux - Brute Force Password Guessing Attempt`
 - **Severity:** `Medium`
 - **MITRE ATT&CK Mapping:** Credential Access (T1110 - Brute Force / Password Guessing)
 - **Status:** Enabled

2. Rule Logic & Parsing Architecture

Under the **Set Rule Logic** tab, paste the following optimized KQL query. This query dynamically parses nested auth strings to aggregate brute force attacks occurring within sliding timelines:

```
Syslog
| where SyslogMessage contains "Failed password"
| extend User = extract(@"for (?:invalid user )?(\\S+)", 1, SyslogMessage)
| extend IPAddress = extract(@"from ([0-9\\.]+)", 1, SyslogMessage)
| summarize FailedAttempts=count() by User, IPAddress, Computer, bin(TimeGenerated, 10m)
| where FailedAttempts >= 3
```

KQL OPTIMIZATION INSIGHT:

The `extract()` function uses specialized regular expressions to isolate variables directly from unformatted syslog strings. Grouping events using `bin(TimeGenerated, 10m)` bundles matching attacks efficiently, preventing alert fatigue in enterprise environments.

3. Advanced Alert Enhancements & Contextual Entity Mapping

Map structural components to top-level Microsoft security entities so that incident engines can auto-correlate cross-platform attacks:

- **Entity Mapping Matrix:**
 - **IP Type:** Bind Address to `IPAddress` column
 - **Account Type:** Bind Name, FullName, and DisplayName properties to `User` column
- **Custom Details Mapping (Metadata Extraction):**
 - `Username` → `User`
 - `SourceIP` → `IPAddress`

- **Attempts** → **FailedAttempts**
- **MachineName** → **Computer**

- **Alert Details Customization:** Set Display Name pattern to: **Failed login attempt detected on host: LinuxVM**

4. Scheduling & Incident Configuration Profile

- **Run Query Every:** **5 minutes**
- **Lookup Data From Past:** **10 minutes**
- **Event Grouping:** Select **Group all events into a single alert** (To maintain low noise profiles).
- **Incident Settings Tab:** Ensure **Create incidents from alerts triggered by this analytics rule** is explicitly toggled to **Enabled**.
- Proceed past Automated Response and select **Review + Create** to save the logic rule.

Phase 5: Enterprise Alert Notification & Escalation Engineering

To guarantee that analysts respond immediately to critical security incidents, configure automated notification routing rules.

1. Navigate to the active **Incidents Page** inside Microsoft Sentinel/Defender.
2. Click on **Email Notification Configuration** → Select **Add incident notification rule**.
3. Under configuration parameters, provide clear classification data:
 - **Rule Name:** **SOC Escalation - Medium Severity Linux Alerts**
 - **Routing Preference:** Send only one alert per incident.
 - **Source Filter Conditions:** Match explicitly against **Microsoft Sentinel**.
 - **Alert Severity Filter:** **Medium**
 - **Recipient Email:** Add your target operational security email inbox.
4. Click **Send test email** to verify connection validity with the notification engine, then click **Submit**.

STRATEGIC SECURITY SUMMARY:

Your lab ecosystem is now fully hardened. When external threat actors attempt brute force entry on your Linux server, the Azure Monitor Agent streams authentication logs to the workspace. Microsoft Sentinel parses the unstructured text, extracts entities, opens a high-visibility security Incident, and sends an alert directly to your inbox within minutes.